Módulo `web`

> Código: `src/web.h`, `src/web.cpp` · Relacionados: 07 — Protocolo e interfaces, protocolo, 08 — App web

Papel no sistema

Transporte do app web: cria o **Access Point** Wi-Fi do colete, serve a página embutida na flash (HTTP, porta 80) e mantém o **WebSocket** (porta 81) por onde passam dados, configuração, avisos e comandos. O conteúdo das mensagens é responsabilidade de protocolo.

Por que Access Point e página embutida: o colete não depende de internet, roteador nem computador da faculdade. O celular conecta direto na rede do colete e abre `http://192.168.4.1`. A página não usa CDN (o celular está sem internet nessa rede).

Bibliotecas: `WebServer` (já vem no core Arduino-ESP32) + `WebSocketsServer` (links2004, versão fixada no `platformio.ini`). A `ESPAsyncWebServer` foi evitada por ter forks e versões confusas.

Dependências

- Usa: `WiFi`, `WebServer`, `WebSocketsServer`, protocolo, config (`WIFI_SSID`, `WIFI_SENHA`, `HTTP_PORTA`, `WS_PORTA`).
- É usado por: `main.cpp` (`web_iniciar`, `web_atualizar`, `web_enviarDados`, `web_enviarConfig`, `web_evento`), `protocolo.cpp` (`web_evento`), `console.cpp` (`web_ativo`, `web_ip`, `web_clientes`).
- Roda em: **loop**. Não é compilado no simulador: `sim/sim_main.cpp` define versões próprias das funções `web_*` que imprimem o JSON no terminal.

Tipos e constantes

Nome	Tipo	Descrição
`index_html`, `manifest_json`, `icon_svg`	`const char[]`	arquivos de `src/web/` embutidos pelo `board_build.embed_txtfiles` (terminam em `'\0'`)
`http`	`WebServer`	servidor HTTP na porta 80
`ws`	`WebSocketsServer`	servidor WebSocket na porta 81
`ativo`	bool	AP no ar e servidores iniciados
`ip[16]`	char[]	IP do AP em texto (`"-"` se não subiu)
`buf[640]`	char[]	mensagem de saída (dados, config, evento); só o loop usa

Os arquivos embutidos viram símbolos do linker com o caminho no nome: `_binary_src_web_index_html_start` etc. Por isso **renomear ou mover um arquivo de `src/web/` exige mudar o `platformio.ini` e as declarações `extern` daqui**.

Rotas HTTP

Método e caminho	Resposta	Tipo
`GET /`	`index.html`	`text/html; charset=utf-8`
`GET /index.html`	`index.html`	`text/html; charset=utf-8`
`GET /manifest.json`	`manifest.json`	`application/manifest+json`
`GET /icon.svg`	`icon.svg`	`image/svg+xml`
qualquer outro	`302` para `http://<ip>/`	—

As quatro rotas servidas levam `Cache-Control: no-cache` (o 302 do `onNotFound` não): depois de gravar um firmware novo, o celular não fica com a página antiga. O manifest e o ícone permitem "Adicionar à tela inicial" (a página abre como app).

Funções

`void web_iniciar()`

- **O que faz:** `WiFi.persistent(false)` (não grava a config do Wi-Fi na flash) → modo AP → `softAP(WIFI_SSID, WIFI_SENHA)` → potência 11 dBm → rádio sem economia de energia → registra as rotas → inicia HTTP e WebSocket → heartbeat → imprime `[wifi] AP "BioSpine" ... -> abra http://192.168.4.1`.
- **Se o AP falhar:** imprime `[wifi] falha ao criar o Access Point` e retorna com `ativo = false`. O colete continua detectando e vibrando; só fica sem app.
- **Por que 11 dBm:** menos potência = menor pico de corrente (ajuda a não resetar a placa quando o motor liga junto) e ainda sobra alcance para uma sala.
- **Por que `setSleep(false)`:** sem economia de energia no rádio, os dados chegam ao app com menos atraso.
- **Chamada por:** `setup`.

`void web_atualizar()`

- **O que faz:** `http.handleClient()` e `ws.loop()`. É dentro de `ws.loop()` que `aoEventoWs` é chamado.
- **Chamada por:** `loop`, a cada volta.

`void web_enviarDados(const Status& s)`

- **O que faz:** `{"t":"data"}` para todos os clientes. Sem cliente conectado, nem monta o JSON.
- **Chamada por:** `loop`, a cada `PERIODO_ENVIO_MS` (10 Hz).

`void web_enviarConfig()`

- **O que faz:** `{"t":"config"}` para todos os clientes.
- **Chamada por:** `app_definirConfig` e `drenarEventos` (eventos com `EVT_CONFIG`, como o fim da calibração).

`void web_evento(const char* nivel, const char* msg)`

- **O que faz:** `{"t":"evt"}` para todos os clientes.
- **Chamada por:** `drenarEventos`, `app_calibrar`, `protocolo_comando`.

`static void aoEventoWs(uint8_t cliente, WStype_t tipo, uint8_t* payload, size_t len)`

- **O que faz:** callback da biblioteca. Na conexão (`WStype_CONNECTED`), ou num texto que `protocolo_comando` responda com `true` (`get`), envia o config **só para aquele cliente**. Texto = comando do app. Outros tipos (desconexão, ping/pong, binário) são ignorados.

`static void servir(const char* tipo, const char* conteudo)`

- **O que faz:** responde `200` com um arquivo embutido e o cabeçalho `no-cache`.

`bool web_ativo()` · `const char* web_ip()` · `int web_clientes()`

- **O que fazem:** AP no ar?; IP em texto; quantos clientes no WebSocket (usados pelo `diag`).

Heartbeat

`ws.enableHeartbeat(5000, 2000, 2)`: ping a cada 5 s; se o pong não chegar em 2 s, duas vezes seguidas, o cliente é desconectado. Sem isso, um celular que saiu da rede continuaria contando como conectado e receberia envios que só travam o loop.

Armadilhas e cuidados

- **A página vai inteira para a flash e para o celular a cada carga** (~85 KB, sem compressão). Comentários e código no `index.html` custam flash e tempo de carga.
- **`softAP` aceita poucos clientes ao mesmo tempo** (o padrão do ESP32 é 4). Cada cliente recebe os dados a 10 Hz.
- **Não existe servidor DNS:** só funciona quem digitar o IP. O redirecionamento `302` só pega caminhos errados no próprio IP (não é um "captive portal" completo).
- **Android "sem internet":** é preciso tocar em "Manter conexão" e, se a página não carregar, desligar os dados móveis (o Android manda o tráfego pelo 4G).
- **Nunca chamar `web_*` da tarefa de controle**: `broadcastTXT` pode bloquear quando um cliente some.